

Assignment Report

The Assignment report submitted in partial fulfillment of the
requirements for
the award of the degree of

The Professional Master Of Embedded And Cyber-physical Systems



ECPS 216 - F'24: IOT Systems & Software

Light Up Your Life

(Measuring the RGB color intensity in the Light)

Under the guidance of
Prof. Quoc-Viet Dang

Name of Candidates

Sahil Kakadiya

UCI Student ID

93282494

Abstract

This Assignment, titled "Light Up Your Life," demonstrates the use of an ESP8266 microcontroller and a TCS34725 RGB color sensor to monitor ambient light intensity and control various LEDs accordingly. The objective is to measure light intensity and provide real-time visual feedback using LEDs to indicate different levels of brightness. The system is divided into three functional parts:

The system operates in four distinct modes, controlled by the pushbuttons on the ZedBoard:

- **Threshold-Based LED Activation:** If the measured light intensity exceeds a set threshold, a blue LED (Onboard) is activated to indicate high light levels.
- **Multilevel Intensity Indicators:** Three LEDs (green, yellow, and red) represent different light intensity levels: low, medium, and high. Each LED is activated based on predefined ranges for average intensity.
- **Dynamic Flashing Based on Intensity:** If the light intensity surpasses the threshold, the blue LED (Onboard) flashes at a rate proportional to the measured intensity, increasing the frequency as the light level rises.

A push-button allows switching between these modes, enabling users to explore different functionalities without reprogramming the device. This implementation showcases an understanding of microcontroller programming, sensor interfacing, and real-time feedback control. The system is designed to be versatile and adaptable, making it suitable for various IoT applications that require real-time light intensity monitoring and indication.

Candidate's Declaration

I hereby certify that the work, which is being presented in the ECPS 216 - F'24: IOT Systems & Software Assignment Report, entitled **Light Up Your Life**, in partial fulfillment of the requirement for the award of the Degree of Master of Embedded System Design and submitted to the institution is an authentic record of my own work carried out during the period **1st Semseter** under the supervision of **Prof. Quoc-Viet Dang**.

I also cited the reference about the text(s)/figure(s)/table(s)/equation(s)/ listing(s) from where they have been taken.

The matter presented in this Assignment report has not been submitted elsewhere for the award of any other degree or diploma from any institutions.

Date: _____

Signature of the Candidate

This is to certify that the above statement made by the candidate is correct to the best of my knowledge. The Viva-Voce examination of **Sahil Mukeshbhai Kakadiya**, Student of The Professional Master Of Embedded And Cyber-physical Systems has been held on

_____.

Signature of
Research Supervisor(s)

Signature of
Head of the Department

Contents

Abstract	2
Candidate's Declaration	3
1 Introduction	6
2 Components and Hardware Used	7
3 Code Structure and Explanation	8
4 Part Breakdown and Logic	9
4.1 Setup Section	9
4.1.1 Setup Section: Code snippet	9
4.2 Loop Section	10
4.2.1 Loop Section: Code snippet	10
4.3 Part 1: Initial Light Intensity Detection	11
4.3.1 Part 1: Code snippet	12
4.4 Part 2: Multi-LED Indicator for Light Levels	13
4.4.1 Part 2: Code snippet	13
4.5 Part 3: Dynamic Flashing LED for High Intensity	15
4.5.1 Part 2: Code snippet	16
4.6 Additional Functionality: Mode Switching	17
4.6.1 Mode Switching: Code snippet	17
5 Schematic	18
5.1 Schematic Zoom on ESP8266 Part	20
5.2 Schematic Zoom on RGB Sensor Part	20
5.3 Schematic Zoom on Indicaton LEDs Part	21
5.4 Schematic Zoom on Push Button Part	21

List of Figures

1	ESP8266 Micro-Controller and TCS34725 RGB SENSOR	7
2	LEDs and Push-Button	7
3	Part 3: Setup Function Code snippet	9
4	Part 3: Loop Function Code snippet	10
5	Part 1 : READING logic RGB values from the sensor	12

6	Part 1 : DECISION logic Based on Average Intensity value	12
7	Part 2 : READING logic RGB values from the sensor	13
8	Part 2: DECISION logic Based on Average Intensity value	14
9	Part 3 : READING logic RGB values from the sensor	16
10	Part 3: DECISION logic Based on Average Intensity value	16
11	Part 3: Mode Switching: ISR Code snippet	17
12	Whole Schematic	18
13	ESP8266 Micro-Controller Schematic	20
14	RGB Color Sensor Schematic	20
15	Push Button Schematic	21

Listings

1 Introduction

This assignment demonstrates the integration of an RGB light sensor with an ESP8266 microcontroller to detect and respond to light intensity. The project is divided into three parts, each building on the previous one, to utilize LEDs to indicate different light intensity levels as measured by the sensor. This report will detail the code logic, components used, and relevant technical information.

Part 1:

- Connect RGB light sensor to ESP8266.
- Read sensor data every second and display it in the IDE console.
- Turn ON the onboard LED or external LED (Blue Color) (Onboard) if the average light intensity exceeds a set threshold.

Part 2:

- Connect 3 external LEDs (Green, Yellow, Red).
- Use LEDs to indicate light intensity levels from the RGB sensor:

Low - Green ON

Medium - Yellow ON

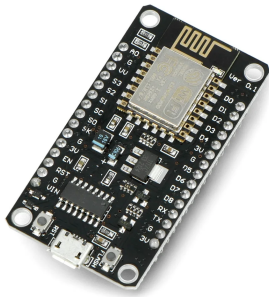
High - Red ON

Part 3:

- Flash the onboard LED or external LED if the average intensity exceeds the threshold.
- Increase the flash rate with higher readings.

2 Components and Hardware Used

1. **ESP8266 Microcontroller:** The core processing unit that manages the sensors, processes the data, and controls the LEDs based on the light intensity.
2. **TCS34725 RGB Light Sensor:** Measures the intensity of red, green, and blue components of ambient light.
3. **LEDs (Red, Yellow, Green, and Blue((Onboard)))):** Measures the intensity of red, green, and blue components of ambient light.
4. **Push Button:** Allows the user to cycle through different operating modes.



(a) ESP8266 NodeMCU kit
12E



(b) TCS34725 RGB color
Sensor

Figure 1: ESP8266 Micro-Controller and TCS34725 RGB SENSOR



(a) Indicator Leds



(b) Push Button

Figure 2: LEDs and Push-Button

3 Code Structure and Explanation

1. Library Imports:

- The **Arduino** library provides the necessary functions for Arduino development.
- **Adafruit_TCS34725** is an Adafruit library specifically designed for the **TCS34725 RGB color sensor**, enabling easy interaction with the sensor.

2. Initialized AND Global Variables:

- The **TCS34725** sensor is initialized with a **50 ms** integration time and a gain of **4x**. These settings are optimal for accurate light data acquisition.
- GPIO pins for **LEDs** and the **push button** are defined, ensuring consistent usage throughout the code.
- These ranges correspond to light intensity levels, with each level triggering a different LED.

3. Initialized AND Global Variables:

- The **TCS34725** sensor is initialized with a **50 ms** integration time and a gain of **4x**. These settings are optimal for accurate light data acquisition.
- GPIO pins for **LEDs** and the **push button** are defined, ensuring consistent usage throughout the code.
- The definitions of the ranges correspond to light intensity levels, with each level triggering a different LED.

4 Part Breakdown and Logic

4.1 Setup Section

the `setup()` function is used to initialize settings and configurations needed for the program to run. It is called once at the beginning of the program

1. The serial communication begins at a baud rate of 9600 for monitoring outputs.
2. All LED pins are set as outputs, and the push button pin is set as input.
 **NOTE: the On-board LED is with the ACTIVE LOW LOGIC (Inverted Logic), means if we write the LOW, the On-board LED Glows.
3. An interrupt is attached to the push button, triggering the modeChange function on a rising edge signal.
4. The program continuously attempts to initialize the TCS34725 sensor and prints a message when successful.
5. Sensor readings are gathered and averaged initially to stabilize and discard any outlier values. A 500 ms delay between readings helps reduce noise.

4.1.1 Setup Section: Code snippet

```
void setup()
{
  Serial.begin(9600);

  /*
  GPD Pin Configuration
  This section initializes the necessary GPD pins with their required settings for the entire assignment.*/
  pinMode(LED_PIN, OUTPUT);
  pinMode(GREEN_LED_PIN, OUTPUT);
  pinMode(YELLOW_LED_PIN, OUTPUT);
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(PUSH_BUTTON_PIN, INPUT);
  /*This call to attachInterrupt() activates the interrupt, allowing the user to change modes*/
  attachInterrupt(digitalPinToInterrupt(PUSH_BUTTON_PIN), modeChange, RISING);
  digitalWrite(LED_PIN, HIGH); // Active LOW logic

  /*Calling the function begin to start the RGB sensor with all the setting given by earlier */
  delay(1000);
  /* Serial print allows a not activate the MPU and our system will not crash by MPU*/
  Serial.print("\n");
  Serial.println("Found RGB Sensor!");

  /*The RGB values are initialized, with the first few readings (e.g., the first 4 to 10 values) potentially containing noise or garbage data.
  This initial setup helps stabilize the readings, allowing accurate and reliable RGB sensor values for subsequent processing.*/
  for (int i = 0; i < 4; i++)
  {
    Serial.println("setting initial value...");
    tcs.getRGB(&red, &green, &blue); // This function return normalized rgb value (0-255)
    red += red;
    green += green;
    blue += blue;
    delay(500);
  }

  red /= 4;
  green /= 4;
  blue /= 4;
  average = (red + green + blue) / 3;

  Serial.print("R:\t" + String(int(red)) + "\tG:\t" + String(int(green)) + "\tB:\t" + String(int(blue)) + "\tA:\t");
  average = (red + green + blue) / 3;
  Serial.print("Average:\t" + String(int(average)) + "\tThresholdAverage:\t" + String(int(thresholdAverage)) + "\t\n");
}
```

Figure 3: Part 3: Setup Function Code snippet

4.2 Loop Section

In the **loop** function, the RGB values are updated every second. The **currMode** value determines which part of the assignment runs, allowing the user to switch between modes by pressing the button.

4.2.1 Loop Section: Code snippet

```
void loop()
{
    getRGBValues();

    switch (currMode)
    {
        case 1:
        {
            Assignment_Part_1();
            break;
        }
        case 2:
        {
            Assignment_Part_2();
            break;
        }
        case 3:
        {
            Assignment_Part_3();
            break;
        }
        default:
        {
            Serial.println("Please selects the Mode/Part of the Assignment : ");
            break;
        }
    }
    /*The delay(1000) function pauses the procedure for 1 second, ensuring the loop runs every second.*/
    delay(1000);
}
```

Figure 4: Part 3: Loop Function Code snippet

4.3 Part 1: Initial Light Intensity Detection

The first part involves connecting the TCS34725 RGB light sensor to the ESP8266 and reading the color intensity every second. When the average light intensity exceeds a predefined threshold (**thresholdAverage**), the blue LED (OnBoard) is activated. The logic is straightforward:

- If the intensity is above the threshold, the blue LED (OnBoard) turns on.
- If not, the LED remains off.

Logic Flow:

1. Read RGB values from the sensor.
2. Calculate the average intensity.
3. Check if the average exceeds the thresholdAverage.
4. Control the blue LED(Onboard) accordingly.

4.3.1 Part 1: Code snippet

```
/* *****  
 * PROCEDURE: Reading the RGB values -- A Todo list that each of the part of the assignment is going to be used  
 * PURPOSE: It reads the RGB sensor Values and average of all.  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 ***** */  
void getRGBValues()  
{  
  tcs.getRGB(&red, &green, &blue); // This function return normalized rgb value (0-255)  
  Serial.print("R:\t" + String(int(red)) + "\tG:\t" + String(int(green)) + "\tB:\t" + String(int(blue)) + "\n");  
  average = (red + green + blue) / 3;  
  Serial.print("Average:\t" + String(int(average)) + "\tthresholdAverage:\t" + String(int(thresholdAverage)) + "\n");  
}
```

Figure 5: Part 1 : READING logic RGB values from the sensor

```
/* *****  
 * PROCEDURE: Part 1 of the Assignment  
 * PURPOSE: Use LEDs to indicate light intensity levels from RGB sensor:  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 ***** */  
void Assignment_Part_1()  
{  
  Serial.println(" Selecting the part 1 of the Assignment ");  
  /* If the Average light intensity exceeds the specified threshold, the blue LED indicator will light up. */  
  if (average > thresholdAverage)  
  {  
    Serial.println(" Threshold is exceeded Status Indication by Blue color LED");  
    digitalWrite(BLUE_LED_PIN, LOW); // Active LOW Logic  
  }  
  else  
  {  
    digitalWrite(BLUE_LED_PIN, HIGH); // Active LOW Logic  
  }  
}
```

Figure 6: Part 1 : DECISION logic Based on Average Intensity value

4.4 Part 2: Multi-LED Indicator for Light Levels

In this part, three LEDs (Green, Yellow, and Red) are used to indicate low, medium, and high light intensity levels based on the calculated average:

- **Green LED:** Turns on for low intensity.
 - **Yellow LED:** Turns on for medium intensity.
 - **Red LED:** Turns on for high intensity.
- If the intensity is above the threshold, the blue LED (Onboard) turns on.
- If not, the LED remains off.

Logic Flow:

1. Check the range in which the average intensity falls.
2. Turn on the respective LED while turning off the others.

4.4.1 Part 2: Code snippet

```
/* *****  
 * PROCEDURE: Reading the RGB values -- A Todo list that each of the part of the assigment is gonig to be used  
 * PURPOSE: It reads the RGB sensor Values and average of all.  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 * ***** */  
void getRGBValues()  
{  
  tcs.getRGB(&red, &green, &blue); // This function return normalized rgb value (0-255)  
  Serial.print("R:\t" + String(int(red)) + "\tG:\t" + String(int(green)) + "\tB:\t" + String(int(blue)) + "\n");  
  average = (red + green + blue) / 3;  
  Serial.print("Average:\t" + String(int(average)) + "\tthresholdAverage:\t" + String(int(thresholdAverage)) + "\n");  
}
```

Figure 7: Part 2 : READING logic RGB values from the sensor

```
/* *****  
 * PROCEDURE: Part 2 of the Assignment  
 * PURPOSE: Use LEDs to indicate light intensity levels from RGB sensor:  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 * ***** /  
void Assignment_Part_2()  
{  
  Serial.println("Selecting the part 2 of the Assignment ");  
  /* If the average light intensity falls within a specified range, the only corresponding LED will light up. */  
  if ((average >= LOW_RANGE_MIN) && (average <= LOW_RANGE_MAX))  
  {  
    Serial.println(" LOW Status Indication by Green color LED ");  
    digitalWrite(GREEN_LED_PIN, HIGH);  
    digitalWrite(YELLOW_LED_PIN, LOW);  
    digitalWrite(RED_LED_PIN, LOW);  
  }  
  else if ((average >= MEDIUM_RANGE_MIN) && (average <= MEDIUM_RANGE_MAX))  
  {  
    Serial.println(" Medium Status Indication by Yellow color LED");  
    digitalWrite(GREEN_LED_PIN, LOW);  
    digitalWrite(YELLOW_LED_PIN, HIGH);  
    digitalWrite(RED_LED_PIN, LOW);  
  }  
  else if ((average >= HIGH_RANGE_MIN) && (average <= HIGH_RANGE_MAX))  
  {  
    Serial.println(" HIGH Status Indication by Red color LED");  
    digitalWrite(GREEN_LED_PIN, LOW);  
    digitalWrite(YELLOW_LED_PIN, LOW);  
    digitalWrite(RED_LED_PIN, HIGH);  
  }  
}
```

Figure 8: Part 2: DECISION logic Based on Average Intensity value

4.5 Part 3: Dynamic Flashing LED for High Intensity

This part builds on Part 1 but adds dynamic flashing functionality to the blue LED (Onboard). The flashing rate increases as the intensity grows higher above the threshold.

- The LED flashes at a rate proportional to the measured intensity, with higher values flashing faster.

Green LED: Turns on for Low intensity.

Yellow LED: Turns on for medium intensity.

Red LED: Turns on for high intensity.

- If the intensity is above the threshold, the blue LED (Onboard) turns on.
- If not, the LED remains off.

Logic Flow: Logic Flow:

1. Check if the intensity exceeds **thresholdAverage**.
2. Map the intensity to a **flashing rate**.
3. Adjust the blue LED's (Onboard) flashing rate accordingly.

****NOTE:**

The sensor value is mapped to the PWM frequency, allowing dynamic adjustment of the LED blinking rate.

As the sensor reading increases, the LED blinks faster, providing real-time feedback based on the intensity.

$\text{thresholdAverage} - 255 = 105; 105/5 = 21;$

Means the rate will be increase by 1 with step of 5 point increase in average value

Writes A pin with the duty cycle of 50 percentage, so that we can see the blinking of the LED

4.5.1 Part 2: Code snippet

```
/* *****  
 * PROCEDURE: Reading the RGB values -- A Todo list that each of the part of the assignment is going to be used  
 * PURPOSE: It reads the RGB sensor Values and average of all.  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 * *****  
 */  
void getRGBValues()  
{  
  tcs.getRGB(&red, &green, &blue); // This function return normalized rgb value (0-255)  
  Serial.print("R:\t" + String(int(red)) + "\tG:\t" + String(int(green)) + "\tB:\t" + String(int(blue)) + "\n");  
  average = (red + green + blue) / 3;  
  Serial.print("Average:\t" + String(int(average)) + "\tthresholdAverage:\t" + String(int(thresholdAverage)) + "\n");  
}
```

Figure 9: Part 3 : READING logic RGB values from the sensor

```
/* *****  
 * PROCEDURE: Part 3 of the Assignment  
 * PURPOSE: Flash (onboard LED) if average intensity exceeds the threshold.  
 * it will Increase flash rate with higher readings when it exceeds.  
 * NAME: Sahil Mukeshbhai Kakadiya  
 * DATE: 10-10-2024  
 * *****  
 */  
void Assignment_Part_3()  
{  
  Serial.println("Selecting the part 3 of the Assignment ");  
  if (average > thresholdAverage)  
  {  
    Serial.println(" Threshold is reached Status Indication ");  
    /*The sensor value is mapped to the PWM frequency, allowing dynamic adjustment of the LED blinking rate.  
    As the sensor reading increases, the LED blinks faster, providing real-time feedback based on the intensity.*/  
    /* thresholdAverage - 255 = 105 ==> 105/5 = 21: Means the rate will be increase by 1 with step of 5 point increase in average value */  
    flashingRateValue = map(average, thresholdAverage, 255, 1, 21);  
    /*analogWriteFreq Writes the determine the freq used based on the Intensity Averaged value */  
    Serial.print("Flashing Rate Value:\t" + String(int(flashesRateValue)) + "\n");  
    /*analogWriteFreq Writes the determine the freq used based on the Intensity Averaged value */  
    analogWriteFreq(flashesRateValue);  
    Serial.println(" Threshold is exceeded Status Indication by Blue color LED with varies its blinking speed");  
    /*analogWrite Writes value to pin with the duty cycle of 50%, so that we can see the blinking of the LED */  
    analogWrite(BLUE_LED_PIN, 50);  
  }  
  else  
  {  
    digitalWrite(BLUE_LED_PIN, HIGH); // Active LOW Logic  
  }  
}
```

Figure 10: Part 3: DECISION logic Based on Average Intensity value

4.6 Additional Functionality: Mode Switching

The push button is used to switch between the three parts/modes:

Mode 1: Executes Part 1.

Mode 2: Executes Part 2.

Mode 3: Executes Part 3.

Logic Flow: Logic Flow:

1. This detachInterrupt reduces the effects of button debounce and prevents multiple a hardware interrupts from being triggered by a single button press..
2. Each button press advances **currMode** to cycle through parts 1, 2, and 3.
3. LEDs are selectively turned off for smooth operation of the current mode.
4. This call to attachInterrupt reactivates the interrupt, allowing the user to change modes again once the debounce effect has settled.

4.6.1 Mode Switching: Code snippet

```

/*=====
 * PROCEDURE: modeChange
 * PURPOSE: This ISR function is triggered when the button is pressed.
 * Each button press advances the program to the next mode (Part 1, Part 2, or Part 3).
 * It cycles through the modes, allowing you to switch parts without reprogramming the ESP8266.
 * NAME: Sahil Mukeshbhai Kakadiya
 * DATE: 10-10-2024
 *=====*/
IRAM_ATTR void modeChange()
{
    /*This detachInterrupt reduces the effects of button debounce and
    prevents multiple a hardware interrupts from being triggered by a single button press.*/
    detachInterrupt(digitalPinToInterrupt(PUSH_BUTTON_PIN));

    /*Each button press advances the program to the next mode.*/
    if (currMode < 3)
    {
        currMode = currMode + 1;
    }
    else
    {
        currMode = 1;
    }

    /*It ensures only the LEDs used in current part/mode is the only one to active and others are going to LOW*/
    if ((currMode == 1) || (currMode == 3))
    {
        digitalWrite(GREEN_LED_PIN, LOW);
        digitalWrite(YELLOW_LED_PIN, LOW);
        digitalWrite(RED_LED_PIN, LOW);
    }
    else
    {
        digitalWrite(BLUE_LED_PIN, HIGH); // Active LOW Logic
    }

    /*This call to delayMicroseconds creates the small delay to for reactivatin the the interrupt, so that the debounce is settled */
    delayMicroseconds(100000);
    /*This call to attachInterrupt reactivates the interrupt, allowing the user to change modes again once the debounce effect has settled.*/
    attachInterrupt(digitalPinToInterrupt(PUSH_BUTTON_PIN), modeChange, RISING);
}

```

Figure 11: Part 3: Mode Switching: ISR Code snippet

5 Schematic

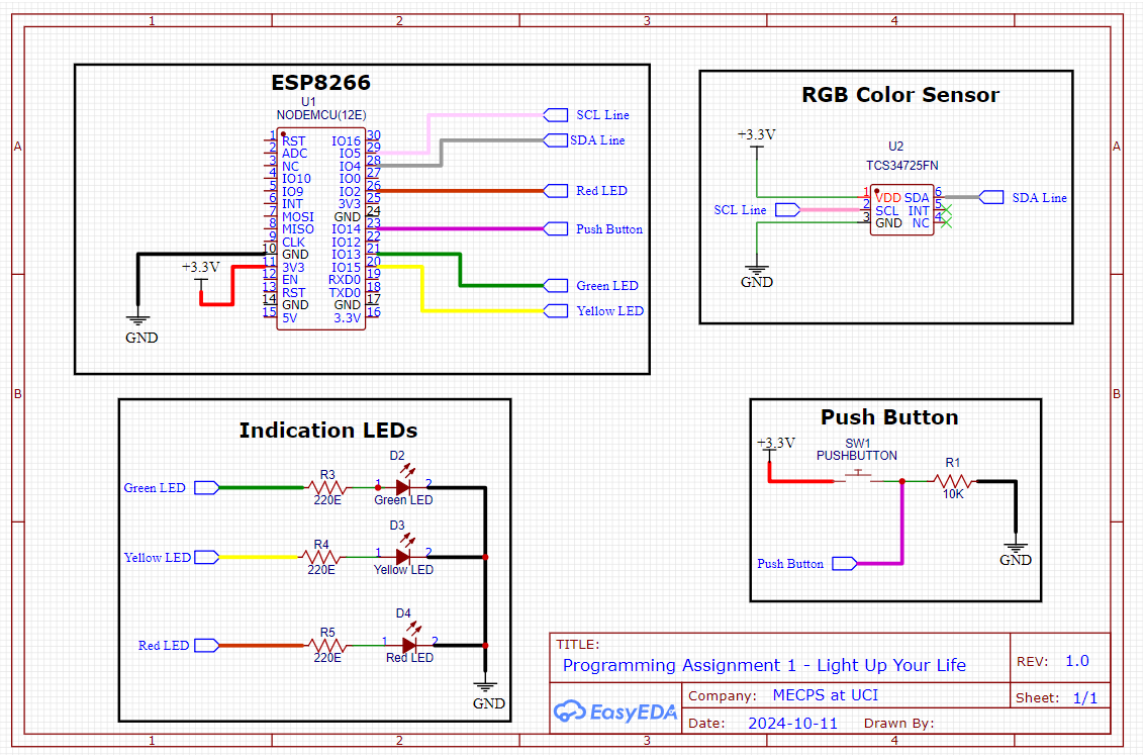


Figure 12: Whole Schematic

****NOTE 1 : SDA and SCL Lines**

RGB sensor uses the I2C communication protocol to transfer the data, so SDA line used to transfer the data and the SCL is clk line to maintain the synchronization.

****NOTE 2 : Labels and Notations**

Net-Lables are Used the to Identify that where the Wires goes from/to where.

Color Codes are used to Identify the Wire -Connection also.

A resistor was placed between the connection of the LEDs and the push-button to prevent overcurrent to the component pins.

5.1 Schematic Zoom on ESP8266 Part

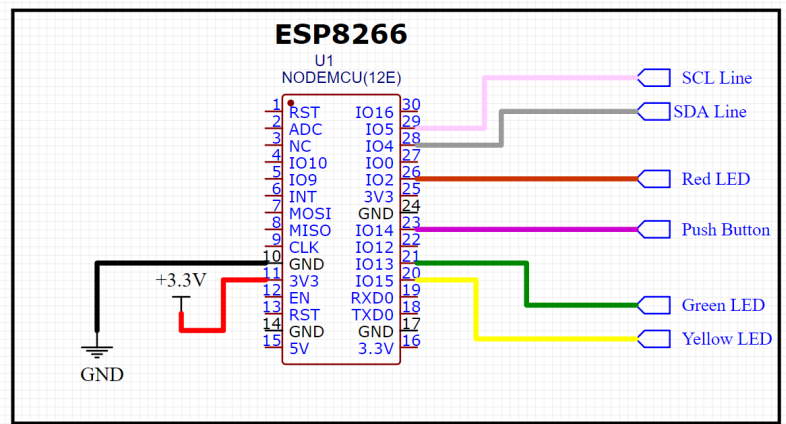


Figure 13: ESP8266 Micro-Controller Schematic

5.2 Schematic Zoom on RGB Sensor Part

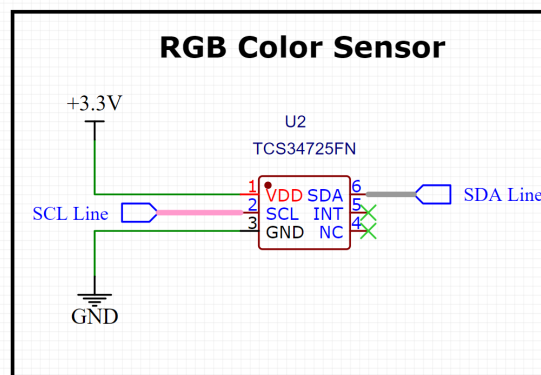
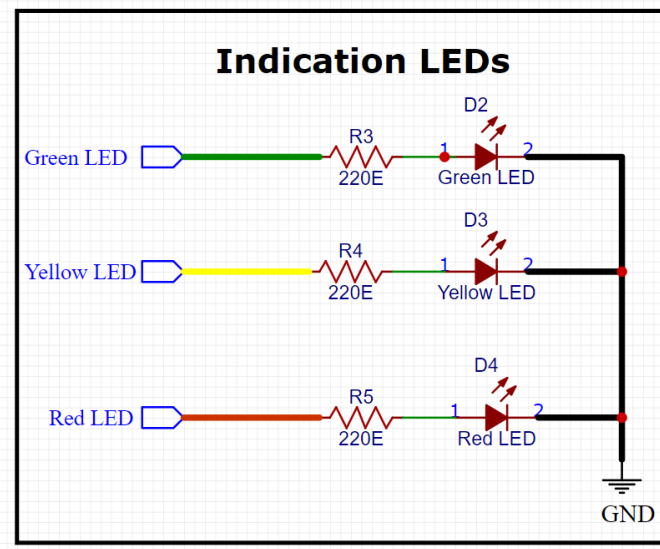


Figure 14: RGB Color Sensor Schematic

5.3 Schematic Zoom on Indication LEDs Part



5.4 Schematic Zoom on Push Button Part

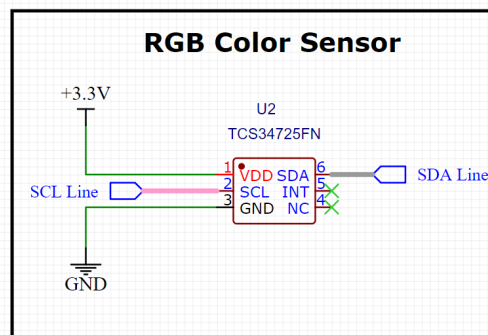


Figure 15: Push Button Schematic